AWS ACADEMY LAB PROJECT

Mini Projet

Application Web Hautement Disponible et Scalable

Salim LAKHAL & Nosaa IYAMU

CSC 8604 — Cloud Computing

Telecom SudParis

Annee universitaire 2025–2026

Table des matieres

1. 1. Contexte et scenario
2. 2. Exigences de la solution
3. 3. Architecture deployee
4. 4. Implementation par phases
5. 5. Securite
6. 6. Code source de l'application
7. 7. Estimation des couts
8. 8. Knowledge Check
9. 9. Enseignements tires
10. 10. Ameliorations possibles
11. 11. Livrables

1. Contexte et scenario

L'**Universite XYZ** se prepare pour la nouvelle annee scolaire. Le service des admissions recoit des plaintes : l'application web de gestion des dossiers etudiants est **lente** ou **indisponible** pendant la periode de pointe des admissions, en raison du nombre eleve de demandes.

Notre mission est de concevoir et deployer une preuve de concept (POC) pour heberger cette application dans le Cloud AWS, en respectant les bonnes pratiques du AWS Well-Architected Framework. L'architecture doit supporter des milliers d'utilisateurs pendant les periodes de pointe.

Application

L'application est un systeme CRUD (Create, Read, Update, Delete) de gestion des dossiers etudiants. Chaque enregistrement contient : nom, adresse, ville, etat, email et telephone.

Composant	Technologie
Backend	Node.js 18 + Express.js
Base de donnees	MySQL 8.0 (Amazon RDS)
Templating	EJS (Embedded JavaScript)
SDK AWS	@aws-sdk/client-secrets-manager v3
Systeme d'exploitation	Ubuntu 22.04 LTS

2. Exigences de la solution

Exigence	Description
Fonctionnelle	Operations CRUD sans latence perceptible

A charge equilibree	Equilibrage du trafic entre serveurs web via Application Load Balancer
Scalable	Auto Scaling selon la demande (CPU target tracking a 50%)
Hautement disponible	Multi-AZ (us-east-1a + us-east-1b), minimum 2 instances actives
Securisee	BDD en sous-reseau prive, Secrets Manager, Security Groups restrictifs
Couts optimises	Instances t3.micro, RDS single-AZ, budget <$100
Performante	Operations rapides sous charge normale, variable et de pointe

3. Architecture deployee

3.1. Schema d'architecture

```
                          INTERNET

                             |

                  +------------v-------------+

                  |  Application Load Balancer |

                  |  Lab-ALB (port 80)         |

                  +-----+-------------+-------+

                        |             |

          +-------------+             +-------------+
```

```
          |                                        |

+---------v-----------+          +------------------v------+

| Public Subnet AZ-1  |          | Public Subnet AZ-2      |

| us-east-1a          |          | us-east-1b              |

|                     |          |                         |

|  EC2 (t3.micro)     |          |  EC2 (t3.micro)         |

|  Node.js + Express  |          |  Node.js + Express      |

+--------+------------+          +-----------+-------------+

         |      Auto Scaling Group           |

         |      min:2  max:4  desired:2      |

         +-----------+     +-----------------+

                     |     |

           +---------v-----v----------+

           | Private Subnet AZ-1      |

           |                          |

           |  Amazon RDS MySQL 8.0    |

           |  db.t3.micro             |

           +--------------------------+



           AWS Secrets Manager (Mydbsecret)

           IAM LabRole via LabInstanceProfile
```

3.2. Ressources AWS

Ressource	Identifiant / Valeur
VPC	`vpc-015f10c548e076bc6` (10.0.0.0/16)
Internet Gateway	`igw-0b4f40e1ae08c1c8e`
Subnet Public AZ-1	`subnet-0c9a86af8623ce46b` (us-east-1a)
Subnet Public AZ-2	`subnet-0dd7aa0d705aeddb9` (us-east-1b)
Subnet Prive AZ-1	`subnet-09fb98871d85efcdc` (us-east-1a)
Subnet Prive AZ-2	`subnet-0f8e7e531bd61e31f` (us-east-1b)
Security Group Web	`sg-063f2429eee0eb830` (ports 80, 22)
Security Group DB	`sg-0e8e5a2d8a5f7b3d4` (port 3306 depuis Web SG)
Security Group ALB	`sg-0522cafcbcdd82dc0` (port 80 depuis Internet)
RDS Endpoint	`lab-db.cpe8i6ikwz9f.us-east-1.rds.amazonaws.com`
ALB DNS	`Lab-ALB-937683819.us-east-1.elb.amazonaws.com`
AMI App Server v2	`ami-00e8094efe00b85cf`
Launch Template	`lt-0db39dbf8cf32b055`

4. Implementation par phases

Phase 1 — Planification et estimation des couts

Objectif : Planifier l'architecture cible et estimer les couts de fonctionnement.

- Creation du schema architectural multi-tier (serveurs web + base de donnees separees)

- Identification des services AWS necessaires : VPC, EC2, RDS, ALB, ASG, Secrets Manager
- Estimation des couts sur 12 mois via AWS Pricing Calculator : environ $51/mois ($611/an)
- Validation de la faisabilite dans le budget du lab ($100)

Phase 2 — Application web fonctionnelle de base (POC)

Objectif : Application fonctionnelle sur une seule instance EC2.

- Creation du VPC (10.0.0.0/16) avec sous-reseaux publics et prives
- Attachement d'un Internet Gateway et configuration de la table de routage
- Lancement d'une instance EC2 POC (t3.micro, Ubuntu 22.04) dans le sous-reseau public
- Installation de Node.js 18, Express, MySQL local sur l'instance unique
- Deploiement de l'application CRUD avec base de donnees locale
- Test reussi : acces via IP publique, operations CRUD fonctionnelles

Limites de cette phase

Credentials BDD en dur dans le code, aucune haute disponibilite, point de defaillance unique.

Phase 3 — Decouplage des composants

Objectif : Separer la base de donnees du serveur web.

- Amazon RDS MySQL 8.0 (db.t3.micro) deploye dans les sous-reseaux prives

- DB Subnet Group couvrant 2 Availability Zones
- Security Group DB : port 3306 autorise uniquement depuis le SG des serveurs web
- AWS Secrets Manager : secret `Mydbsecret` contenant host, user, password, database, port
- Application modifiee pour utiliser `@aws-sdk/client-secrets-manager`
- Nouveau serveur web avec IAM LabInstanceProfile (LabRole)
- Migration des donnees de MySQL local vers RDS via `mysqldump`

Phase 4 — Haute disponibilite et scalabilite

Objectif : Architecture multi-AZ avec equilibrage de charge et auto-scaling.

- Application Load Balancer deploye sur 2 AZ, health check sur `/` toutes les 30 secondes
- AMI personnalisee (v2) avec Node.js 18, application, dependencies npm et service systemd
- Launch Template base sur l'AMI v2 : type t3.micro, SG web, IAM LabInstanceProfile
- Auto Scaling Group : min=2, max=4, desired=2, Target Tracking (CPU 50%)
- Application accessible via le DNS de l'ALB

5. Securite

5.1. Isolation reseau

- VPC dedie avec sous-reseaux publics (serveurs web) et prives (base de donnees)
- RDS dans des sous-reseaux prives sans acces Internet direct

5.2. Security Groups (principe du moindre privilege)

Security Group	Regles entrantes
ALB SG	Port 80 (HTTP) depuis 0.0.0.0/0
Web Server SG	Port 80 depuis ALB SG + Port 22 (SSH)
Database SG	Port 3306 depuis Web Server SG uniquement

5.3. Gestion des secrets

- Credentials BDD stockees dans AWS Secrets Manager (secret `Mydbsecret`)
- Application recupere les credentials au demarrage via l'AWS SDK
- IAM Role LabRole attache aux instances EC2 via LabInstanceProfile
- Aucun credential en dur dans le code source

5.4. Protection des donnees

- Toutes les requetes SQL utilisent des requetes parametrees (protection contre les injections SQL)
- Principe du moindre privilege applique a tous les Security Groups

6. Code source de l'application

6.1. Structure du projet

```
/home/ubuntu/app/

  |-- package.json       # Dependencies: express, mysql2, ejs, @aws-sdk

  |-- app.js             # Serveur Express principal

  |-- views/
```

```
        |-- index.ejs     # Page d'accueil (liste des etudiants)

        |-- add.ejs       # Formulaire d'ajout

        |-- edit.ejs      # Formulaire de modification
```

6.2. Routes de l'application

Route	Methode	Action
`/`	GET	Affiche la liste de tous les etudiants
`/add`	GET	Affiche le formulaire d'ajout
`/add`	POST	Insere un nouvel etudiant en base
`/edit/:id`	GET	Affiche le formulaire de modification
`/edit/:id`	POST	Met a jour un etudiant existant
`/delete/:id`	GET	Supprime un etudiant

6.3. Flux de demarrage

1. Le service systemd `student-app.service` lance `node app.js` au boot
2. `getSecret()` recupere les credentials depuis AWS Secrets Manager
3. Connexion a RDS MySQL, creation de la table `students` si inexistante
4. Le serveur Express ecoute sur le port 80

6.4. Evolution du code entre les phases

Aspect	Phase 2 (POC)	Phases 3-4 (Production)
Base de donnees	MySQL local	Amazon RDS
Credentials	En dur dans le code	AWS Secrets Manager
SDK AWS	Aucun	@aws-sdk/client-secrets-manager

IAM	Aucun	LabInstanceProfile (LabRole)
Demarrage	Manuel	Automatique (systemd)
Securite reseau	SG simple	SG DB isole en sous-reseau prive

7. Estimation des couts

Service	Configuration	Cout/mois	Cout/an
Amazon EC2	2x t3.micro On-Demand (730h)	$15.33	$184
Amazon RDS	db.t3.micro MySQL, 20 GB GP2, single-AZ	$17.63	$211
Application Load Balancer	1 ALB + LCU estimees	$16.20	$200
AWS Secrets Manager	1 secret, ~1000 appels/mois	$0.40	$5
Data Transfer	~10 GB sortant/mois	$0.90	$11
Total estime		**~$51/mois**	**~$611/an**

Optimisations possibles

Reserved Instances (1 an, no upfront) : reduction d'environ 40% sur EC2 et RDS, soit un total annuel d'environ $365. Savings Plans offrent une flexibilite similaire.

8. Knowledge Check

Score obtenu : **80% (40/50)** — seuil requis : 70% (35/50) — **Reussi**.

Question	Theme	Resultat
Q1	Architecture VPC et sous-reseaux	Correct (10/10)
Q2	Migration BDD (mysqldump)	Incorrect (0/10)
Q3	AMI + Launch Config + ASG	Correct (10/10)
Q4	DNS de l'ALB pour acces	Correct (10/10)
Q5	Diagnostic SG + serveur web	Correct (10/10)

9. Enseignements tires

1. **Approche par phases** — Commencer par un POC simple puis complexifier progressivement permet de valider chaque etape avant d'avancer.
2. **Decouplage web/BDD** — La separation des couches est essentielle pour le scaling independant et la haute disponibilite.
3. **Gestion des secrets** — AWS Secrets Manager est une bonne pratique par rapport au hardcoding des credentials.
4. **AMI + systemd** — Une AMI pre-configuree avec auto-start garantit que les instances ASG sont immediatement operationnelles.
5. **ALB + ASG** — La combinaison fournit automatiquement haute disponibilite et scaling horizontal.
6. **Gestion du budget** — Avec un budget de $100, le choix t3.micro + RDS single-AZ est un bon compromis cout/fonctionnalite.

10. Ameliorations possibles

Amelioration	Service AWS	Benefice
HTTPS	ACM + ALB HTTPS Listener	Chiffrement en transit
Multi-AZ RDS	RDS Multi-AZ	Failover automatique de la BDD
CDN	Amazon CloudFront	Latence reduite pour assets statiques
Authentification	Amazon Cognito	Controle d'acces utilisateur
Infrastructure as Code	CloudFormation / Terraform	Reproductibilite et versioning
Monitoring	CloudWatch + SNS	Alertes operationnelles
CI/CD	CodePipeline + CodeDeploy	Deploiement automatise

11. Livrables

Livrable	Statut
Schema architectural	Inclus (presentation slide 4 + rapport section 3)
Estimation des couts	Incluse (presentation slide 6 + rapport section 7)
Infrastructure AWS deployee	Operationnelle
Application web fonctionnelle	Accessible via ALB
Knowledge Check	80% (40/50) — Reussi
Presentation PowerPoint	13 slides
Rapport	Ce document

Rapport redige le 13 mars 2026

Salim LAKHAL & Nosaa IYAMU — Telecom SudParis